

Documentación Algoritmos **MEMORIA**

Nombre Asignatura: Algoritmos y estructuras de datos
Profesor: Aleix Lopez
Curso: 2025/2026
Autor/es: Pau Alvarez



Índice/Index

1.- Descripción de los ADTs implementados.....	2
1.1 MemoryNode.....	3
1.2 Vector.....	3
1.3 List — Lista enlazada simple.....	3
1.4 DLList — Lista doblemente enlazada.....	3
1.5 CircularList y CircularDLList.....	3
1.6 CircularVector.....	4
1.7 MHVector — Vector de cabeza movable.....	4
1.8 Stack — Pila (LIFO).....	4
1.9 Queue — Cola (FIFO).....	4
2.- Análisis comparativo de rendimiento.....	4
2.1 Análisis previo: costes a priori.....	4
2.2 Medición empírica.....	5
3.- Algoritmo de ordenación.....	6
3.1 Mediciones.....	7
4.- Port a c++ y encapsulación de datos.....	8
4.1 Encapsulación.....	8
4.2 Ocultación de implementación.....	8
4.3 Rule of Five.....	8
4.4 Otras ventajas del port.....	8
5.- Los ADTs como base de la orientación a objetos.....	9



1.- Descripción de los ADTs implementados

Todos los ADTs (Tipos de Datos Abstractos) siguen un patrón arquitectónico similar. Están compuestos por un `*struct*` que contiene los datos de estado y un campo `*ops_*` que apunta a una tabla de punteros a función, que representa la API pública. La creación de un ADT se realiza mediante una función como ``XXX_create(capacity)``. Todas las interacciones con el ADT se llevan a cabo a través de ``adt->ops_->operacion(...)``.

1.1 MemoryNode

Un `*MemoryNode*` es la unidad básica de almacenamiento para todos los ADTs. Almacena un puntero genérico `*void* *data_*`, el tamaño en bytes del dato y punteros `*next_*` y `*prev_*` para uso en listas. Es el único ADT que interactúa directamente con el gestor de memoria EDK. Su API incluye: `setData*` (almacena una referencia), `memCopy` (copia profunda), `memConcat` (concatena datos), `softReset` (suelta el dato sin liberarlo), `reset` (libera el dato y limpia), `softFree` (libera el nodo sin liberar el dato), `free` (libera ambos)

Esto permite que los algoritmos sean independientes del tipo de dato almacenado.

1.2 Vector

Un Vector es un almacén de tamaño fijo con acceso $O(1)$ por índice. Internamente utiliza un array contiguo de `MemoryNode` pre-reservado en la creación. Mantiene `head_*` y `tail_*` como índices del primer elemento válido y del primer hueco libre. Los puntos fuertes del Vector son el acceso aleatorio $O(1)$, la inserción y extracción en el final en $O(1)$, y una excelente localidad de caché. Sin embargo, los puntos débiles son la inserción y extracción en el principio, que requieren desplazar todos los elementos y tienen una complejidad de $O(n)$.

1.3 List — Lista enlazada simple

Una Lista enlazada simple es una estructura en la que cada elemento vive en un `MemoryNode` asignado dinámicamente en el heap, enlazado con `next_*`. La estructura guarda punteros directos al primero y al último para que las operaciones en los extremos sean $O(1)$. Los puntos fuertes de la Lista son la inserción y extracción en el principio y en el final en $O(1)$. Los puntos débiles son la extracción en el final, que es $O(n)$, y el acceso por posición, que es $O(i)$, además de una peor localidad de caché que el Vector.

1.4 DLList — Lista doblemente enlazada

Una Lista doblemente enlazada es una extensión de la lista simple que añade el enlace `prev_*` en cada nodo. Esto convierte la extracción en el final en $O(1)$ y permite que el acceso a una posición y la extracción en una posición arranquen desde el extremo más cercano a la posición, reduciendo el recorrido a la mitad. Los puntos fuertes de la DLList son que todas las operaciones en extremos son $O(1)$ y el acceso a una posición es mejor que en la lista simple en la mitad trasera. Los puntos débiles son la necesidad de un puntero extra por nodo y el recorrido hasta una posición sigue siendo $O(i)$.

1.5 CircularList y CircularDLList

Las listas circulares son variantes de las listas que no tienen un final explícito. La iteración utiliza la longitud como contador de parada. Estas listas son útiles para planificadores round-robin, buffers de reproducción continua o cualquier problema que requiera recorrido indefinido sin condición de fin.



1.6 CircularVector

Un CircularVector es una variante circular del Vector que utiliza índices `head_` y `tail_` que envuelven módulo `capacity_`. Combina el acceso contiguo en memoria con la capacidad de rotar los elementos $O(1)$ avanzando los índices. Es una base natural para implementar una cola eficiente.

1.7 MHVector — Vector de cabeza movable

Un MHVector se crea con el doble de capacidad real solicitada, situando `head_` en el centro del bloque. Esto permite la inserción en el principio en $O(1)$ amortizado, ya que solo se decrementa el índice de cabeza sin mover ningún elemento. La ventaja es que la inserción en el principio pasa de $O(n)$ a $O(1)$ amortizado. La desventaja es que hasta la mitad de la memoria puede estar desaprovechada, y cuando el espacio delantero se agota y hay que crecer, el coste real de esa operación es $O(n)$.

1.8 Stack — Pila (LIFO)

Una pila es un almacén LIFO en el que el último elemento insertado es el primero en salir. Se implementa sobre un array contiguo de `MemoryNode`. El tope siempre está en `storage_[tail_ - 1]`. Todas las operaciones características son $O(1)$.

1.9 Queue — Cola (FIFO)

Una cola es un almacén FIFO en el que el primer elemento insertado es el primero en salir. Se implementa como un array circular para que las operaciones de encolar y desencolar sean $O(1)$. Los índices avanzan con $(\text{index} + 1) \% \text{capacity_}$. El campo `length_` evita la ambigüedad entre cola llena y vacía.

2.- Análisis comparativo de rendimiento

2.1 Análisis previo: costes a priori

Antes de realizar cualquier medición, el análisis teórico utilizando la notación Big-O predice el comportamiento de cada estructura. Esto se llama análisis asintótico. Es una herramienta útil que nos permite elegir la estructura de datos adecuada sin tener que medir nada, solo conociendo cómo se va a acceder a los datos.

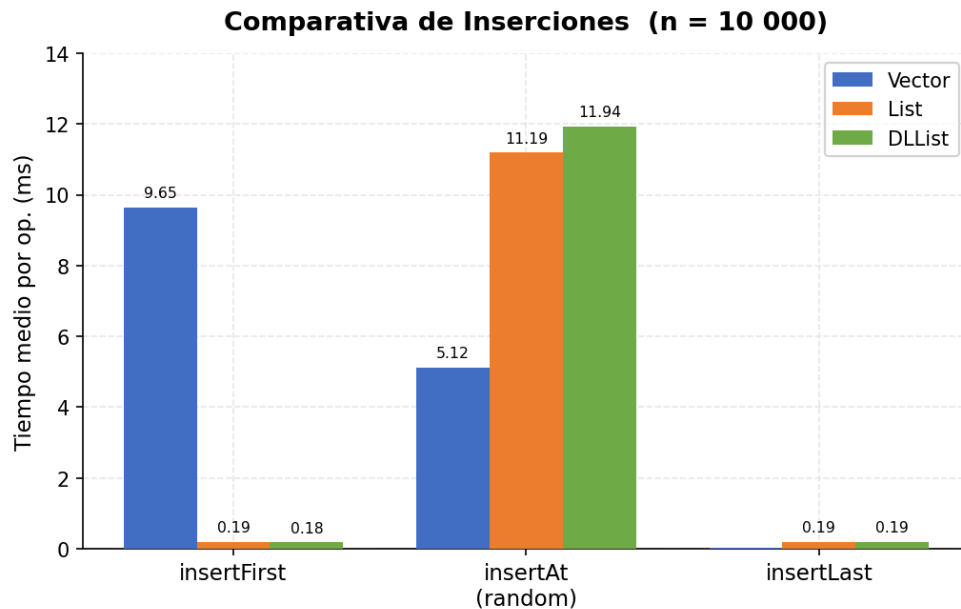
La notación Big-O no tiene en cuenta las constantes multiplicativas ni cómo afecta el caché. Por eso, es necesario hacer mediciones reales para confirmar o afinar los resultados. Esto nos da una idea más clara de cómo se comportará la estructura de datos en la práctica.

Hipótesis: creo que el Vector será muy lento cuando se trata de agregar o eliminar elementos al principio, pero muy rápido cuando se trata de agregar o eliminar al final. En cambio, la Lista debería ser bastante lenta cuando se trata de eliminar elementos del final. La lista doblemente vinculada (DLList) debería ser bastante equilibrada y no destacar en nada en particular, es decir, no será la más rápida ni la más lenta en ninguna de las operaciones.



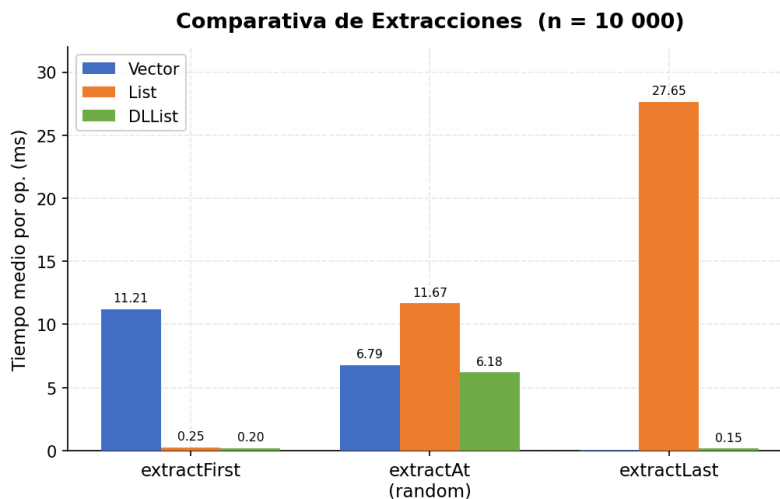
2.2 Medición empírica

Las mediciones se realizan en el proyecto PR12_Comparative usando QueryPerformanceCounter de Windows (resolución de microsegundos). Cada operación se ejecuta sobre $n = 10\,000$ elementos y se reporta el tiempo medio por operación individual.



El Vector tarda mucho más que las listas al insertar al principio. Esto se debe a que el Vector necesita mover todos los elementos, lo que demora 50 veces más. En cambio, cuando se inserta en una posición aleatoria, las listas pueden parecer más rápidas en teoría. Sin embargo, en la práctica, las listas suelen ser más lentas. Esto se debe a que los nodos de la lista están distribuidos en diferentes partes de la memoria. Por otro lado, el Vector almacena todos sus elementos en un bloque contiguo de memoria. Esto hace que sea más rápido acceder a los elementos, ya que éstos están uno al lado del otro. En las listas, cada vez que se accede a un elemento, es posible que el ordenador deba buscarlo en otra parte de la memoria. Esto puede causar un retardo. En resumen, el Vector es más rápido para insertar elementos al principio. Sin embargo, al insertar en una posición aleatoria, el Vector sigue siendo más rápido debido a su forma de almacenar los elementos.

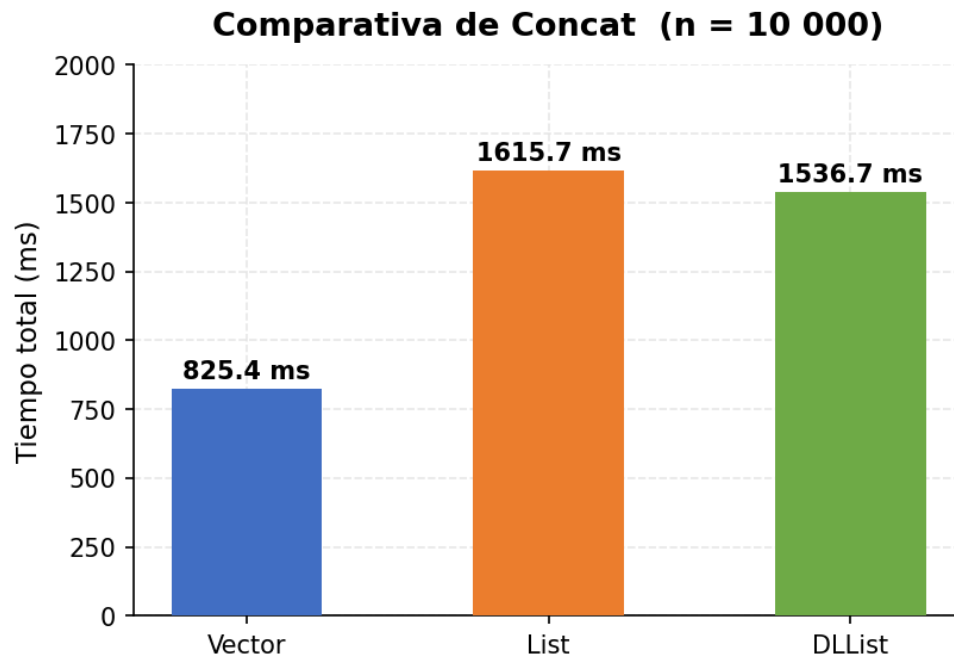
Extracciones:





La diferencia más llamativa es `extractLast`: la List tarda 27.65 ms/op frente a 0.16 ms de DLList (factor $\times 173$). Esto demuestra empíricamente el coste $O(n)$ del recorrido en lista simple para encontrar el penúltimo nodo, frente al $O(1)$ que permite el enlace `prev_`. La DLList justifica aquí su overhead de un puntero extra por nodo.

Concat:



Los tres tienen una complejidad de tiempo de $O(n)$, pero el Vector es aproximadamente 2 veces más rápido debido a cómo se almacenan los datos en la memoria. Esto se debe a que copiar un bloque de datos contiguos en la memoria es más eficiente porque aprovecha mejor el caché del procesador. En cambio, seguir punteros dispersos en el montón de memoria es más lento.

3.- Algoritmo de ordenación

Quicksort es un método para ordenar un conjunto de datos. Funciona de la siguiente manera:

1. Se selecciona un elemento base.
2. Se reorganiza el conjunto de datos: los elementos menores o iguales al base van a un lado, y los mayores van al otro.
3. Se repite este proceso con cada parte del conjunto de datos.

Este método utiliza una técnica llamada partición de Lomuto. En esta técnica, el elemento base es el que está más a la derecha en cada parte del conjunto de datos. El método es muy flexible y puede ordenar cualquier conjunto de datos, sin importar el tipo de datos que contenga.



3.1 Mediciones

Las mediciones se realizan con QueryPerformanceCounter sobre 10 repeticiones por punto, reportando la media. Se prueban tres escenarios de entrada para evidenciar la diferencia entre el caso promedio y el peor caso:

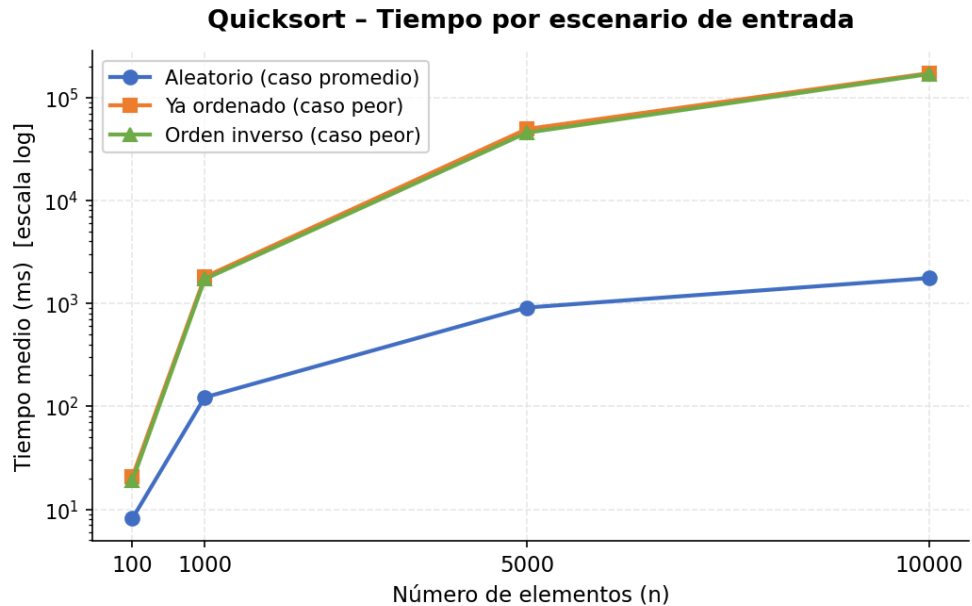


Fig. 4 — Quicksort: tiempo por escenario (escala logarítmica)

La gráfica en escala logarítmica hace visible la bifurcación entre $O(n \log n)$ y $O(n^2)$: con entrada aleatoria el crecimiento es suave, mientras que la entrada ordenada o invertida dispara el tiempo hasta 172 s para $n=10\,000$ — un factor $\times 97$ respecto al caso promedio.

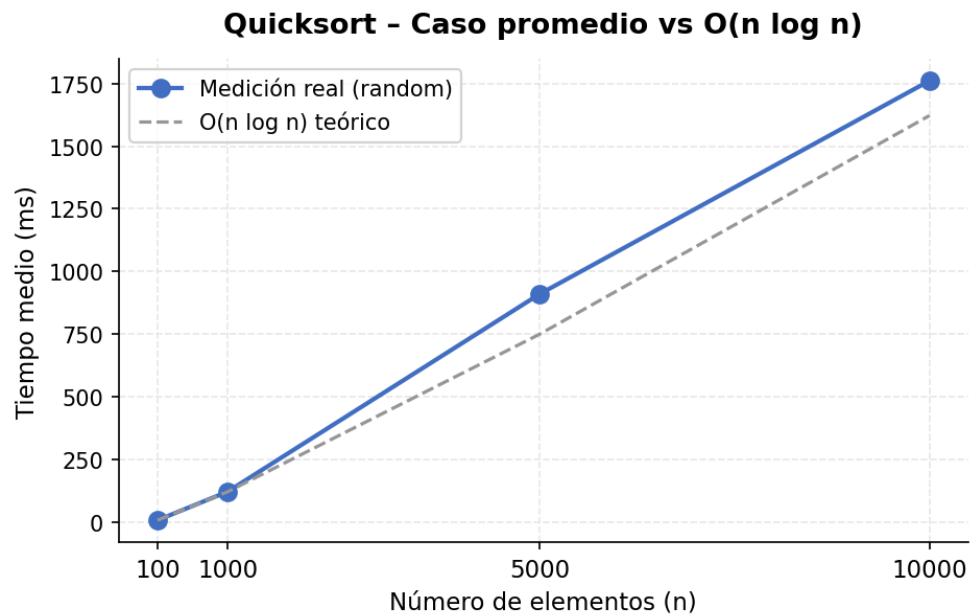


Fig. 5 — Quicksort caso promedio vs $O(n \log n)$ teórico



Con entrada aleatoria la curva medida sigue casi exactamente la curva $O(n \log n)$ teórica ajustada al primer punto, validando empíricamente el caso promedio del algoritmo.

4.- Port a c++ y encapsulación de datos

Los ADTs MemoryNode, Vector, List, DLLList, Stack y Queue han sido trasladados a C++17. Este traslado no utiliza herencia ni funciones virtuales; en su lugar, se enfoca en un diseño orientado a datos utilizando clases simples.

4.1 Encapsulación

En C, todos los campos de una estructura son públicos. Esto significa que cualquier parte del código puede acceder y modificar directamente campos como `list->length_` o `queue->tail_`, lo que puede romper las invariantes internas sin que el compilador advierta de ello. Sin embargo, en C++, los campos son privados.

Gracias a esto, el compilador impide el acceso a campos como `head_`, `tail_` o `storage_` desde fuera de la clase. El estado interno de estos objetos solo se puede modificar a través de los métodos públicos, los cuales se aseguran de mantener las invariantes (por ejemplo, `length_ <= capacity_`, `tail_ < capacity_`).

4.2 Ocultación de implementación

Para el cliente, solo es visible la interfaz pública declarada en el archivo `.h`. El hecho de que la `Queue` utilice internamente un array circular es un detalle de implementación que permanece oculto. Si en el futuro se decide cambiar a una lista enlazada, el código cliente no necesitará recompilarse siempre y cuando la interfaz no sufra cambios.

4.3 Rule of Five

En C, el programador debe recordar manualmente llamar a `destroy` para liberar recursos; de lo contrario, se produce una fuga de memoria. En C++, el destructor se invoca automáticamente cuando un objeto sale de su ámbito de visibilidad. Se han implementado los cinco operadores de la “Regla de los Cinco” para gestionar adecuadamente copias y movimientos de objetos.

El constructor de movimiento es especialmente útil, ya que permite transferir la propiedad de un objeto grande en tiempo constante (solo se mueven punteros), en lugar de copiar todos los datos.

4.4 Otras ventajas del port

- const correctness: Los métodos de consulta se marcan como `const`, lo que evita que un método de lectura modifique el estado del objeto accidentalmente.

- explicit: El constructor de un solo argumento se declara como `explicit` para evitar conversiones implícitas no intencionadas.

- Placement new: Los arrays de `MemoryNode` se construyen con `new (&storage_[i]) MemoryNode()` dentro de la memoria gestionada por el EDK, sin invocar el operador `new` global y manteniendo la compatibilidad con el gestor de memoria de la práctica.



5.- Los ADTs como base de la orientación a objetos

La programación orientada a objetos no se limita a lenguajes como C++ o Java. Es un patrón de diseño que se puede aplicar en cualquier lenguaje que permita el uso de punteros a funciones. Un ejemplo práctico de esto son los tipos de datos abstractos implementados en C.

En la programación orientada a objetos, una clase es similar a un struct que contiene datos y un campo llamado ops_ que apunta a una tabla de operaciones. Los métodos son funciones registradas en esta tabla ops_. El constructor y el destructor son equivalentes a funciones como XXX_create y ops_->destroy. La encapsulación se logra por convención, ya que se accede al estado interno solo a través de ops_, tratando los campos como si fueran privados, aunque el lenguaje no lo exija. El polimorfismo se consigue porque dos tipos de datos abstractos diferentes pueden exponer la misma interfaz, como List y DLList, que tienen el mismo conjunto de operaciones y pueden usarse de forma intercambiable. La herencia no existe de forma directa, pero la composición de structs permite extender el comportamiento de forma similar.

C++ simplemente eleva estas convenciones al nivel del compilador, lo que permite una verificación estática, una gestión automática del ciclo de vida y una sintaxis más concisa. Sin embargo, el modelo mental sigue siendo el mismo: separar lo que se hace de cómo se hace.

Implementar tipos de datos abstractos en C obliga a entender la programación orientada a objetos en su esencia, sin la sintaxis simplificada de los lenguajes de alto nivel. La tabla de punteros a función es como una tabla de funciones manuales, y el campo ops_ es un puntero a esta tabla. Portar el código a C++ es básicamente dejar que el compilador gestione lo que antes se hacía manualmente.



6. BIBLIOGRAFÍA

- 1.- CuriousWalk. (2021, 2 de marzo). Quick Sort Algorithm [Video]. YouTube. <https://www.youtube.com/watch?v=WprjBK0p6rw>
- 2.- Wikipedia contributors. (2024). Sorting algorithm. https://en.wikipedia.org/wiki/Sorting_algorithm
- 3.- cppreference.com. (2024). C++ Standard Library reference. <https://en.cppreference.com>
- 4.- Portfolio Courses. (2021, 24 de diciembre). Quicksort Algorithm Implementation | C Programming Example [Video]. YouTube. <https://www.youtube.com/watch?v=0jDiBM68NGU>
- 5.- Encapsulamiento - EcuRed. (s.f.). EcuRed. <https://www.ecured.cu/Encapsulamiento>
- 6.- M, G. P. (2023, 21 de abril). Explicación de la notación Big O con Ejemplos. freeCodeCamp.org. <https://www.freecodecamp.org/espanol/news/explicacion-de-la-notacion-big-o-con-ejemplo/>
- 7.- Implementación del ADT racional (C++). (s.f.). Fundamentos de la Programación Orientada a Objetos. <https://fundamentospoorrr.blogspot.com/p/implementacion-del-adt-racional-c.html>
- 8.- Infogram. (s.f.). Infogram. <https://infogram.com/es/crear/grafico>